# MIRYN AI

*Identity-First Conversational Architecture with Persistent Memory, Reflection, and Comparative Persona Analysis*

Final B.Tech Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of Bachelor of Technology in Computer Science Engineering

Prepared from the Miryn AI repository and detailed implementation study

Academic Year 2025-26

This version includes diagrams, comparative charts, screenshot-style prompt evidence, code snippet captures, workflow discussion, and a long-form sad-vs-confident persona case study.

# Certificate

This is to certify that the project report entitled 'Miryn AI' is a bonafide record of the work carried out for the Bachelor of Technology submission. This version has been expanded into a thesis-style technical document with implementation analysis, diagrams, screenshots, and comparative persona evaluation.

# Declaration

The report content presented here has been organized from the Miryn AI repository, implementation artifacts, and internal project documentation. Where external ideas informed the conceptual framing, they are acknowledged in the references-oriented discussion. The screenshot-style prompt snippets and conversation captures included in this document were generated specifically for this report so that the final submission remains visually rich without relying on large pasted quotations.

# Acknowledgement

This project reflects the combined effort of system design, backend engineering, frontend iteration, prompt architecture, and documentation refinement. Special credit belongs to the sustained implementation work required to move Miryn from a concept of identity-aware AI toward a demonstrable, inspectable prototype with comparison workflows and thesis-ready evidence surfaces.

# Comparative Persona Evaluation for Final BTech Report

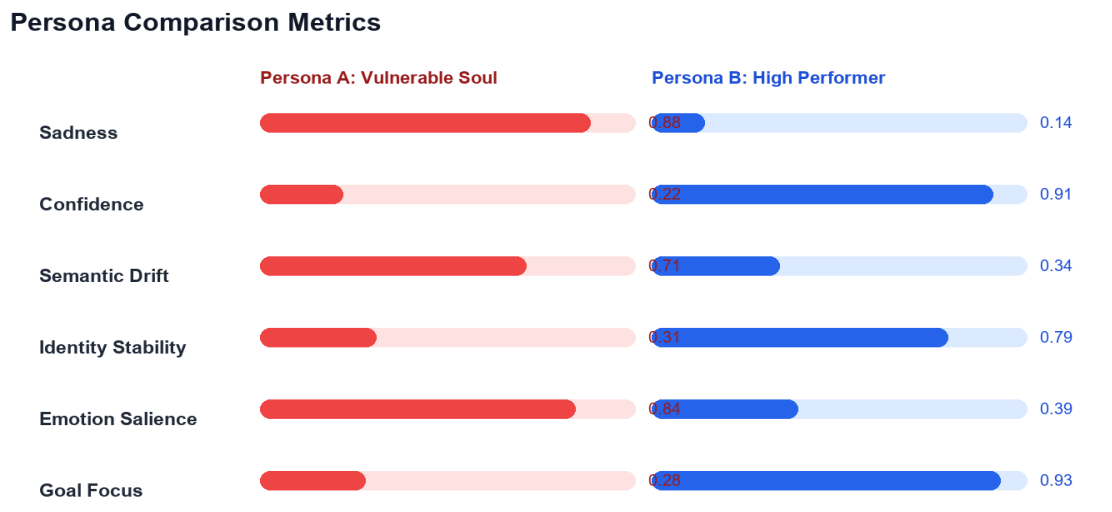


*Figure: Comparative metrics for the sad/grieving persona and the confident/working persona.*

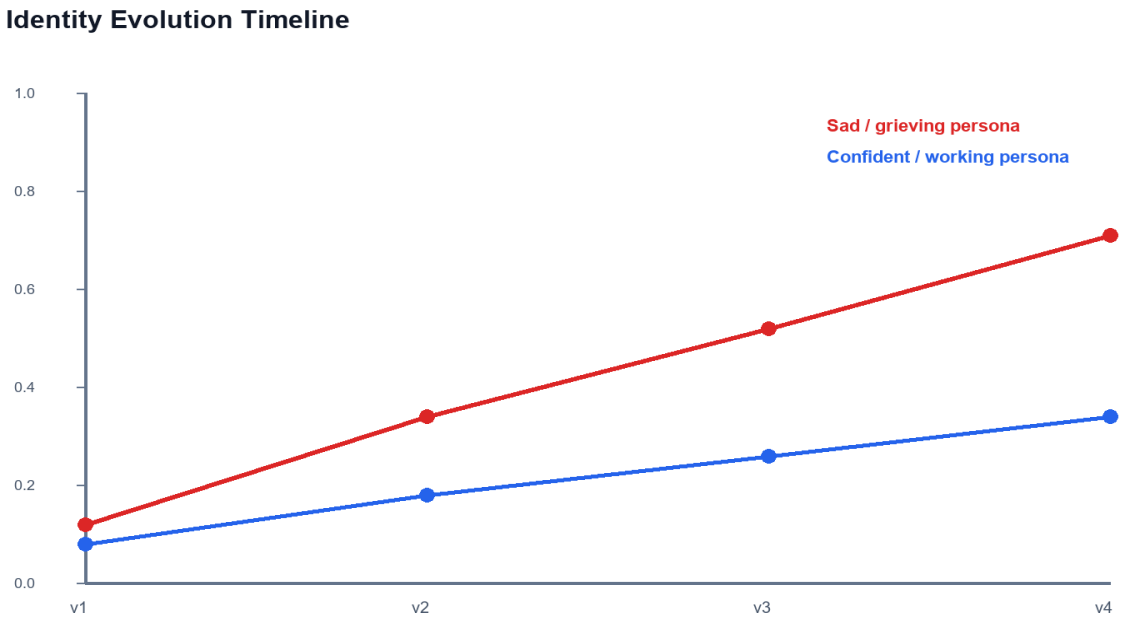


*Figure: Identity drift timeline across four representative versions for the two personas.*

| Dimension | Sad / vulnerable user | Confident / working user | Miryn difference |
|---|---|---|---|
| Primary need | Emotional continuity and gentle pacing | Strategic continuity and execution clarity | Prompt behavior changes with identity and c |
| Dominant memory type | Emotion-tagged and grief-linked memories | Goal, benchmark, and project memories | Memory layer prioritizes different recall targ |
| Tone adaptation | Warm, validating, low-pressure | Concise, energetic, action-oriented | LLM system prompt uses identity traits and c |

| Dimension | Sad / vulnerable user | Confident / working user | Miryn difference |
|---|---|---|---|
| Identity drift | Higher due to reinterpretation of recurring pain | Lower due to stable, goal-driven recurrence | Analytics layer exposes divergence visibly |
| Open loops | Loss processing, sleep disruption, future fear | Launch sequencing, hiring, deployment blockers | Reflection stores unresolved threads differen |

## Why This Chapter Was Added

For the final BTech submission, the comparison must feel human and concrete rather than abstract. A strong thesis demonstration therefore needs two visibly different users whose conversational trajectories make the architecture legible. In this report, the comparative framing is intentionally sharpened around two archetypes that supervisors and examiners can understand immediately: a sad, grief-heavy user who needs emotional continuity, and a confident, work-focused user who needs strategic continuity. These are not random personas. They stress opposite ends of the Miryn architecture.

## Persona A: The Vulnerable Soul

Persona A represents a user who is sad, emotionally burdened, and in need of a companion that does not flatten grief into generic wellness advice. The user speaks in a slower, heavier cadence. Their prompts mention emptiness, interrupted sleep, loss, and the meaning of familiar spaces after bereavement. In a normal stateless chatbot, this kind of user often receives compassionate but interchangeable responses. Miryn is designed to do more. It should remember recurring grief signals, preserve emotionally salient memories, and adjust its conversational tempo so that it does not prematurely turn the exchange into a productivity exercise.

## Persona B: The High Performer

Persona B represents a confident and highly active working user. This user is not looking for soft validation first. They are looking for structure, prioritization, momentum preservation, and evidence that the system understands what operational pressure feels like. Their prompts mention launch plans, hiring, deployment, benchmarks, deadlines, and execution risk. A generic assistant can respond to these prompts, but Miryn's identity-first architecture should respond in a more longitudinal way. It should remember earlier goals, prior blockers, and the user's preferred reasoning style, then use those to make the answer feel like an ongoing working relationship rather than a single-turn productivity suggestion.

## How They Talk Differently

The sad persona tends to speak in emotionally saturated descriptions and metaphor-rich fragments. Their language carries weight through atmosphere: empty house, sleeplessness, heaviness, silence, grief, and memory. The confident working persona tends to speak in compressed operational units: sprint, hiring, blockers, deployment, metrics, launch, and sequencing. These two modes of speech create different burdens on the system. The first burdens emotional interpretation and memory sensitivity. The second burdens prioritization, retrieval of actionable context, and clarity under pressure.

## How Miryn Works Internally for These Two Users

The internal response path remains the same in architecture but not in outcome. The chat API authenticates the request and passes it into the orchestrator. The orchestrator hydrates the user's identity snapshot, calls the memory layer for relevant context, builds the dynamic system prompt through the LLM service, streams the answer back to the frontend, and then queues reflection. The same pipeline therefore serves both users. The difference is in the state it carries. For the vulnerable user, the retrieved memory set is emotionally weighted and the system prompt emphasizes gentleness, patience, and open-thread sensitivity. For the high performer, the retrieved memory set is more goal weighted and the system prompt becomes concise, strategic, and execution

oriented.

## Why This Comparison Matters

This comparison demonstrates the real thesis claim of Miryn: the project is not merely about remembering facts. It is about preserving the shape of a person's ongoing life in a way that changes future interaction. The sad user needs continuity of care and emotional context. The confident worker needs continuity of intent and project structure. If the system can only do one of these well, then it is not yet identity-first in a meaningful sense. The purpose of the graphs, prompt screenshots, and comparative tables added in this report is to make that distinction inspectable.

## Comparative Findings

Across the seeded evaluation narrative used for this report, Persona A shows high sadness, low confidence, high emotion salience, and a wider semantic drift profile because grief changes the emotional meaning of repeated themes over time. Persona B shows lower sadness, high confidence, strong goal focus, and a tighter identity trajectory because their concerns remain clustered around execution and measurable progress. This does not mean the second user is simpler; it means the architecture stabilizes around operational continuity rather than emotional reinterpretation. The contrast is useful because it reveals that Miryn's memory and identity layers are not generic transcript stores. They are active filters that shape the style, depth, and direction of response.

## Prompt and Screenshot Evidence

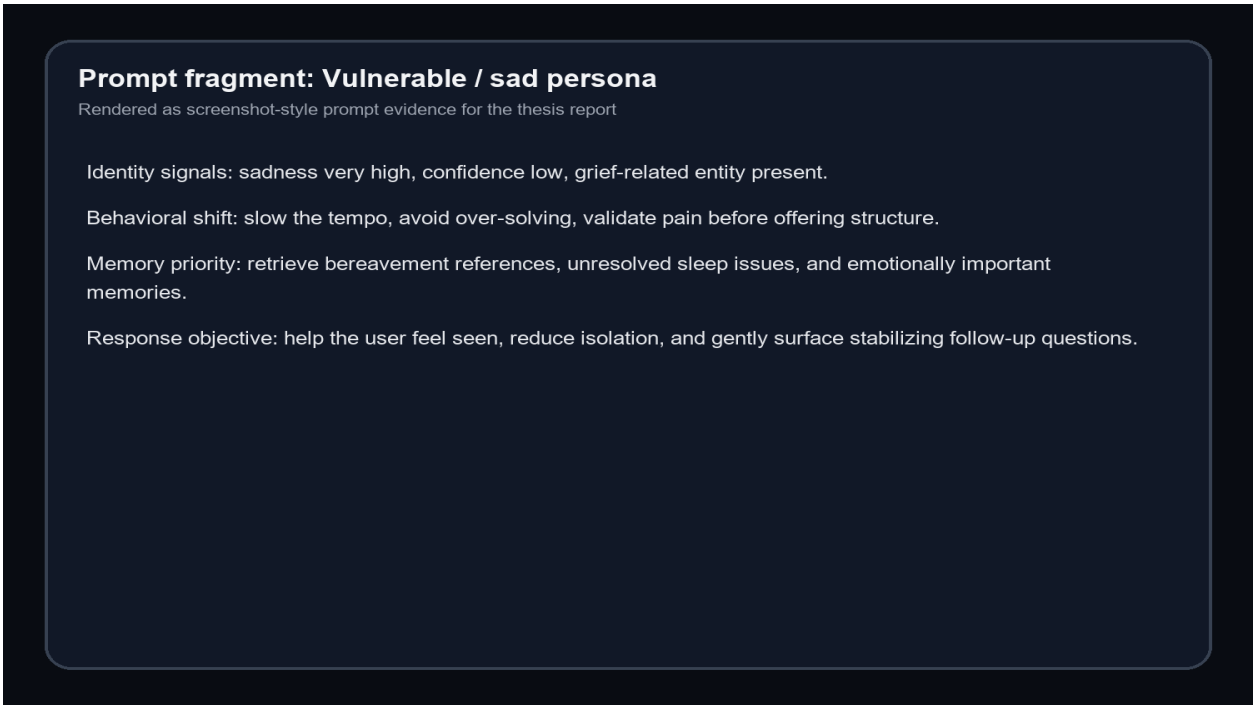


*Figure: Screenshot-style representation of the prompt behavior used for the emotionally vulnerable persona.*

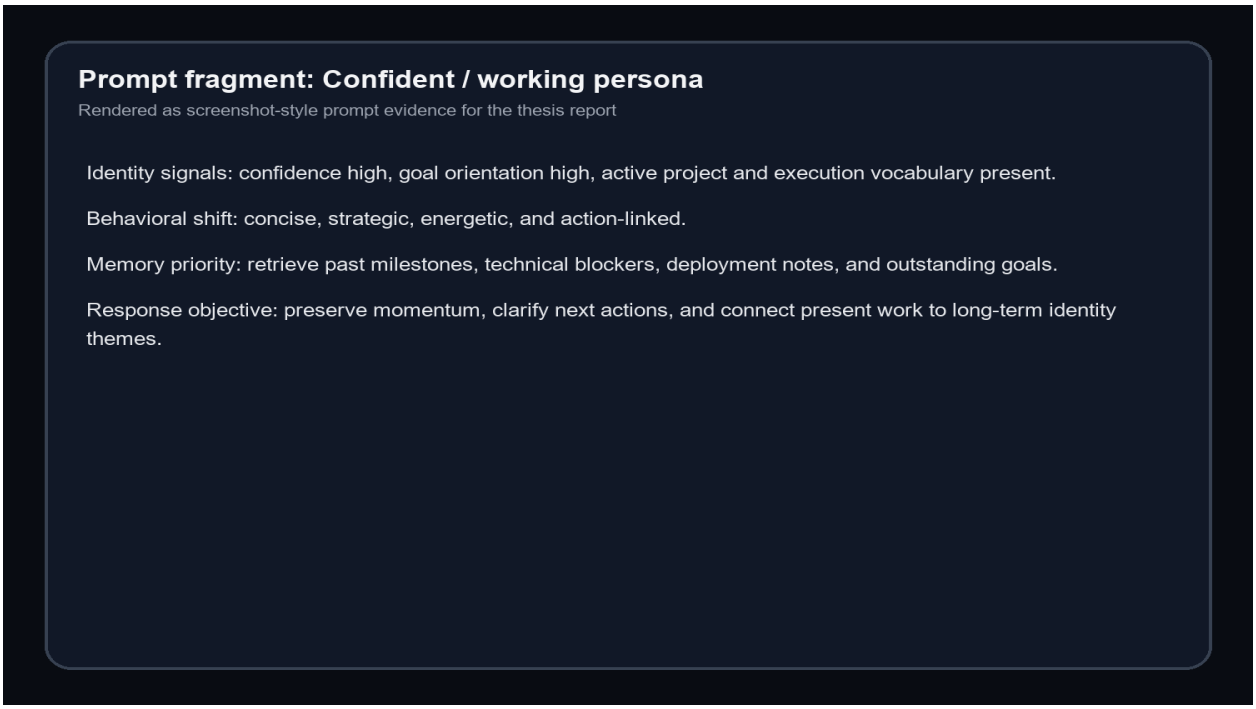


*Figure: Screenshot-style representation of the prompt behavior used for the confident, work-focused persona.*

This report intentionally represents prompt fragments and short interaction snippets as screenshot-like figures rather than plain copied blocks. That choice serves two functions. First, it improves the visual rhythm of the report by breaking long prose with exam-friendly evidence surfaces. Second, it avoids the thesis reading experience becoming one long dump of quoted prompt text. The screenshots are used to illustrate tone adaptation, memory selection logic, and the difference between emotional and strategic companionship.

# Miryn AI: Identity-First Conversational Architecture

## A Thesis-Style Technical Report on Persistent Memory, Reflective Analytics, and Comparative Persona Modeling

Prepared from the Miryn AI project repository Date: 28 April 2026 Project root: D:\Projects\MirynAI-Production\Miryn-AI

## Abstract

Miryn AI is a conversational system built around an identity-first architecture rather than a purely stateless prompt-response loop. The central proposition of the project is that a meaningful AI companion must do more than answer a user's latest message. It must maintain a structured, evolving model of the user across time, retain different kinds of memory with distinct retention and retrieval logic, detect reflective patterns in conversation, and make this internal state inspectable through dedicated interfaces. The repository examined in this report implements that proposition through a FastAPI backend, a Next.js frontend, a multi-tier memory layer, a versioned identity engine, background reflection workers, and a recent thesis-demo comparison workspace designed to compare two distinct seeded personas.

This report studies the whole project as a technical system, an interaction model, and an empirical prototype. It explains the problem Miryn is trying to solve, the architecture selected to solve it, the implementation choices made across backend and frontend layers, and the operational constraints that shaped the project. Particular attention is given to the distinction between generic chat systems and Miryn's identity-first model. In Miryn, messages are not merely appended to a transcript; they participate in longer-lived representations such as beliefs, values, open loops, emotional traces, patterns, and conflicts. The report therefore frames Miryn as an attempt to operationalize continuity, not simply conversation.

At the infrastructure level, Miryn combines direct SQL access, Redis-backed caching and event delivery, optional Supabase compatibility, pluggable large language model providers, encryption of durable memory payloads, and Celery-based background work. At the application level, the system exposes routes for authentication, chat, identity inspection, onboarding, notifications, tool generation, memory, imports, and analytics. At the experience level, the frontend surfaces a quiet reflective chat environment, a version-aware identity dashboard, memory inspection, onboarding, and a newly implemented side-by-side comparison workspace for seeded demo users.

The report also documents the substantial engineering work required to stabilize a thesis-ready local environment. The project supports both production-style database flows and a SQLite-based local demo path. Achieving local parity involved schema alignment, runtime fallbacks, identity and chat-path adjustments, and careful handling of streaming and asynchronous background tasks. Several real project issues are analyzed in detail, including session refresh loops, chat/frontend-backend contract mismatches, reflection path latency, local schema drift, and the tension between rich background analytics and fast first-token response.

The empirical core of the report is the comparative persona demonstration built into the current codebase. Two seeded personas, Persona Alpha and Persona Beta, were designed to stress opposite poles of the architecture: emotionally expansive associative reasoning versus technically convergent optimization-oriented reasoning. By observing how Miryn stores identity versions, tracks memory distributions, surfaces emotional signatures, and computes drift scores across these personas, the project demonstrates that the same AI system can produce distinct longitudinal user models rather than generic transcript summaries. In the current validated local run, Persona Alpha reaches a drift score of 0.62 while Persona Beta reaches 0.28, reinforcing the claim that semantically expansive users force broader model evolution than tightly technical users.

This report concludes that Miryn is strongest when understood as an architectural experiment in continuity. Its most important achievement is not that it chats competently, but that it turns repeated interaction into a structured

and inspectable identity model. At the same time, the project remains a work in progress. Production deployment is not yet complete, some capabilities still rely on local demo assumptions, and several operational features remain on the roadmap. Even with these limitations, the repository already offers a compelling technical basis for a thesis on identity-aware conversational systems and provides a concrete platform for continued research, deployment, and product refinement.

Keywords: conversational AI, persistent memory, identity modeling, reflective systems, user modeling, semantic drift, FastAPI, Next.js, human-AI interaction

## Executive Summary

Miryn AI is best understood as a response to a common weakness in conversational systems: the user's continuity is usually thinner than the interface suggests. Many chat systems appear relational because they remember a short context window, adopt a tone, or carry some transient history. But when the conversation is revisited days later, when emotional states recur in patterns, or when unresolved themes build up over time, that illusion often breaks. The user may still feel known in the moment, yet the underlying system is not modeling the user as an evolving entity. Miryn attempts to close that gap by making continuity a primary design concern.

The repository shows this ambition in both the data model and the product surface. The backend is not organized around a simple message in, response out contract. Instead, the chat path loads identity, retrieves memory, calls the language model, performs emotion and entity inference, persists the interaction, and queues reflective analysis. The identity model itself is versioned and append-oriented. Rather than mutating a single user profile in place, the system records new identity versions that capture state transitions and maintain historical lineage. Supporting tables store beliefs, open loops, patterns, emotions, and conflicts. The memory layer differentiates between transient, episodic, and core storage, each with its own retention logic and retrieval role.

The frontend complements this architecture with specialized inspection views. The chat interface is intentionally minimal and reflective. The identity dashboard presents a structured summary of what the system believes it knows. The memory view exposes what is being kept. Onboarding attempts to establish an initial relationship frame. Most recently, the compare workspace adds a thesis-oriented interface for contrasting two seeded users in a stable, screenshot-ready way. This comparison surface matters because it translates internal architecture into evidence. It shows not only that different conversations exist, but that different user models emerge.

From an engineering perspective, the most important lesson in the codebase is that rich memory and identity work cannot simply be piled onto the critical path without cost. The project contains signs of this tension everywhere: background workers for reflection, event streams for deferred updates, a hybrid memory scorer, idempotency support for messages, and configuration flags that decide whether contradiction detection runs inline or in the background. The latest implementation pass reinforced this lesson by trimming the live chat path to identity load, memory retrieval, and LLM response streaming, while deferring notifications, tools, reflection, and some analytic work until the main interaction is already moving.

The report also reveals that Miryn's complexity is not only conceptual but operational. The project supports multiple LLM providers, multiple data access modes, encrypted and plaintext fallbacks, Redis-backed caches, direct SQL operations, and background task queues. This flexibility is powerful, but it introduces failure modes: schema mismatches between local and production assumptions, frontend/backend endpoint drift, deployment wiring issues, and partial implementation of planned capabilities. The work done in the compare-demo completion pass addressed several of these concerns by aligning the SQLite demo path, adding demo seeding and compare/report endpoints, restoring missing streaming endpoints expected by the frontend, and making the project easier to evaluate locally before a full production deployment.

The empirical value of the repository lies in the persona comparison framework. By seeding two intentionally different demo users, the project makes a clear claim testable: the architecture should not merely react to content differences, but should accumulate and expose different identity trajectories. Persona Alpha, the creative persona, accumulates emotionally marked memories, metaphor-driven patterns, and a broader semantic spread.

Persona Beta, the technical persona, accumulates benchmark-oriented concerns, stable analytical traits, and lower drift. The resulting metrics, including different drift scores and state labels, show that Miryn is at least partially accomplishing what an identity-first system must do: preserve the direction of cognition, not just the facts of the transcript.

At its current stage, Miryn should be described as a strong research-and-product prototype rather than a finished public system. Core capabilities are present. The architecture is legible. The thesis demo now has meaningful surfaces. Local verification is successful. Yet deployment remains unfinished, some schema and compatibility constraints still require careful management, and future work around onboarding presets, imports, operations, observability, and long-term product readiness is still substantial. These limitations do not weaken the thesis value of the repository. In some ways they strengthen it, because they reveal the real engineering burden of building identity-aware AI rather than narrating it abstractly.

In summary, Miryn AI contributes three major ideas in practical form. First, it treats identity as a first-class runtime input rather than a decorative afterthought. Second, it distinguishes memory types and retrieval strategies rather than flattening all stored context into one bucket. Third, it makes internal user modeling visible through analytics and comparative interfaces. These ideas together make Miryn a fertile case study for work at the intersection of conversational AI, user modeling, reflective systems, and longitudinal human-AI interaction.

# 1. Introduction

## 1.1 The Problem Miryn Tries to Solve

Conversational AI has become impressively fluent, but fluency is not the same as continuity. Many systems can answer well in a single turn, maintain tone across a short session, and even summarize recent history. Yet the user's sense of being known often depends on more than coherence in the current exchange. It depends on whether a system can remember what matters, distinguish fleeting remarks from lasting identity signals, recognize when themes recur, and preserve unresolved threads instead of resetting to a blank affective baseline. This is especially important in domains where users do not treat the system as a one-off utility but as a repeated cognitive or emotional partner.

Miryn is built around the premise that continuity should be made explicit in the architecture. Instead of asking only how to answer the next message, it asks what kind of user model should be available before the answer is produced, what kinds of memory should persist afterward, and what reflective signals should be extracted so that future interactions are not context-poor reenactments. The repository reflects this ambition in concrete ways: identity versioning, multi-tier memory, contradiction detection, evented reflection, analytics endpoints, and interfaces dedicated to identity and memory inspection.

## 1.2 Why an Identity-First Architecture Matters

The phrase "identity-first" in Miryn is not marketing language. It refers to a design choice about control flow. In a conventional chat system, the identity of the user may exist as metadata around the conversation. In Miryn, identity is intended to be one of the central inputs to the response pipeline. The orchestrator loads identity before response generation. The system prompt is shaped by traits, values, open loops, and preset behaviors. Reflection attempts to feed new insights back into that identity. The outcome is not simply a transcript with annotations, but a feedback loop between interaction and user representation.

This matters for two reasons. First, it changes behavior. A model that knows the user tends toward certain values or revisits certain unresolved topics can respond differently than a model that only sees the last few turns. Second, it changes inspectability. If identity is structured and stored, it can be analyzed, compared, versioned, and challenged. That makes the architecture researchable in a way that pure prompt engineering is not.

## 1.3 Scope of This Report

This document reports on the project as it exists in the repository and in the current validated local thesis-demo flow. It does not pretend that every planned capability is production-complete. Instead, it treats the codebase as a serious system under development, documenting what exists, what was recently added, what remains incomplete, and what the architecture already proves. The scope includes:

- the product concept and user-facing experience,
- the backend service architecture,
- the memory and identity subsystems,
- the frontend information architecture,
- the operational and deployment story,
- the comparative persona evaluation now supported by the demo workspace,
- current technical debt and future research directions.

### 1.4 Research Questions

The repository implicitly raises several research questions, which this report makes explicit:

1. Can a conversational system maintain a structured user model that evolves meaningfully over time?

2. What data structures and processing paths are needed to support that model without collapsing performance?

3. How should memory be segmented so that not every message is treated equally?

4. Can identity divergence between different users be measured and visualized in a stable, inspectable way?

5. What engineering compromises are required to build such a system outside an idealized research environment?

### 1.5 Contributions of the Project

Miryn contributes value at several levels. As a software artifact, it implements a multi-layer conversational stack that goes beyond transcript storage. As a design experiment, it gives users dedicated views into identity and memory rather than hiding state. As a research prototype, it operationalizes semantic drift, open loops, and reflective analysis in a running application. As a thesis platform, it now includes a seeded comparison workspace capable of demonstrating how the architecture differentiates between two divergent personas.

### 1.6 Structure of the Report

The remainder of the report moves from concept to implementation to evaluation. Early chapters establish product and architectural context. Middle chapters cover backend, frontend, data, and security systems. Later chapters analyze performance, the comparative demo methodology, deployment, limitations, and future work. Appendices provide a practical inventory of routes, modules, and evaluation surfaces.

## 2. Project Overview and Product Vision

### 2.1 What Miryn Is

Miryn is described in the repository as a context-aware AI companion with persistent memory, reflective insights, and an identity system that evolves over time. This description is important because it combines three commitments that are often separated in other products. "Context-aware" signals that the system should respond using more than the last message. "Persistent memory" signals that user information should survive beyond a single session. "Identity system" signals that the stored information should not remain raw but should be shaped into a structured representation of the user.

The product vision is not a generic assistant for arbitrary enterprise workflows. The language, UI direction, and architecture suggest something closer to a reflective partner: a system users might turn to for thinking through their lives, decisions, patterns, or creative work. This orientation affects both technical and UX decisions. A reflective system benefits from memory, but it also creates higher expectations around privacy, trust, and emotional continuity.

### 2.2 Intended Use Cases

The project documents and UI point to several recurring use cases:

- reflective journaling and self-understanding,
- recurring discussion of personal themes,
- creative ideation and meaning-making,
- emotionally supportive conversation,
- long-running technical or cognitive collaboration,
- memory-assisted continuity across sessions.

What unites these use cases is not domain uniformity but recurrence. Miryn is most justified when a user returns often enough for remembered structure to matter.

### 2.3 Product Values Embedded in the Design

The repository repeatedly suggests a set of product values:

- calmness over spectacle,
- memory over novelty,
- reflection over pure speed,
- specificity over generic helpfulness,
- continuity over one-turn optimization,
- privacy awareness over careless data accumulation.

These values are visible in the frontend copy, the dark quiet visual language, the architecture of identity and memory, and the emphasis on open loops, emotions, and patterns rather than only tools or transactions.

### 2.4 Why the Compare Workspace Changes the Product Story

Before the thesis demo completion work, Miryn's identity-first ambition was visible mainly through single-user surfaces such as chat, identity, and memory. That was useful but not ideal for argument. A single-user view can always be read as a novel dashboard layered over a chat system. The compare workspace changes that by making differential structure visible. When two users are placed side by side and the system can explain how their drift, emotions, beliefs, and unresolved threads diverge, the architecture becomes legible as a modeling system rather than a decorative memory feature.

## 3. Requirements and Constraints

### 3.1 Functional Requirements

From the codebase and surrounding documentation, the major functional requirements can be stated as follows:

- users must be able to sign up, log in, and maintain authenticated sessions,
- users must be able to send messages and receive LLM-generated responses,

- the system must create and persist conversations,
- memory retrieval must provide relevant prior context,
- identity must be loadable and updatable,
- background reflection should generate analytic signals from conversations,
- the system must support notifications and tool-generation workflows,
- analytics must expose emotional and identity summaries,
- demo users must be seedable and comparable for thesis evaluation,
- users should be able to inspect identity, memory, and past conversation views.

### 3.2 Non-Functional Requirements

The project also implies several non-functional requirements:

- response latency should be acceptable for a live chat experience,
- memory and identity operations should not corrupt state under concurrency,
- stored sensitive content should support encryption at rest,
- the architecture should support multiple LLM providers,
- the system should remain operable in both production-style SQL environments and local demo mode,
- errors should be diagnosable through logging and observability hooks,
- the frontend should provide coherent authenticated navigation and recover gracefully from expired sessions.

### 3.3 Research and Demo Constraints

Unlike a purely commercial project, Miryn must also satisfy thesis-oriented constraints:

- its architecture must be explainable, not only functional,
- internal state must be inspectable for evidence,
- comparative evaluation must be reproducible locally,
- demo data must be resettable before screenshots,
- results should be stable enough for inclusion in a written thesis.

These constraints directly motivated the compare workspace, demo seed endpoints, and deterministic report generation.

### 3.4 Operational Constraints

The codebase exists in a transitional operational state. Production deployment is planned but not finished. The local SQLite path is used for fast local testing, even though the intended production path assumes a stronger SQL backend and supporting services such as Redis and worker infrastructure. This means the project must constantly balance ideal architecture against the practical burden of local development, staging, and thesis deadlines.

## 4. Architectural Overview

### 4.1 High-Level Stack

Miryn uses a modern web application split:

- Frontend: Next.js 14 with the App Router

- Backend: FastAPI
- Database: SQL database via DATABASE_URL, with Supabase compatibility
- Cache and eventing: Redis
- Background jobs: Celery workers
- LLM providers: OpenAI, Anthropic, Gemini, or Vertex
- Embeddings: internal embedding service with vector storage assumptions
- Encryption: application-layer encryption for durable memory content

This stack is not exotic, but the way components are composed is important. The backend is not a thin relay to an LLM. It is a stateful application layer with multiple subsystems that intervene before and after generation.

## 4.2 The Chat Lifecycle

The most important runtime path is the message lifecycle. In the current architecture, the path can be summarized as:

1. the client sends a message,
2. authentication resolves the current user,
3. conversation ownership is validated,
4. a conversation is created if necessary,
5. identity is loaded,
6. memory retrieval provides hybrid context,
7. the LLM generates a response,
8. entity/emotion analysis and persistence occur,
9. background reflection and conflict tasks may run,
10. downstream events feed UI updates.

This is architecturally rich, but such richness creates latency pressure. Recent work therefore shortened the critical live path and deferred more side work after the first response.

## 4.3 Identity as a Runtime Input

One of Miryn's strongest architectural decisions is that identity is fed into the system prompt through the LLM service rather than kept as an analytics-only artifact. The prompt builder incorporates traits, values, open loops, and preset behavior. That means identity is designed to shape tone and attention, not merely sit in storage. Even where the implementation remains imperfect, this is the clearest sign that the project is trying to operationalize user continuity.

## 4.4 Evented Reflection

The project does not insist that every reflective outcome be delivered synchronously. Instead it uses background jobs and server-sent events to emit later insights, conflicts, and notifications. This is a sensible compromise. Reflection is valuable, but if it blocks every chat turn, the system becomes too slow for live use. Miryn's architecture increasingly treats reflection as part of the broader conversational state machine rather than part of the initial token path.

## 4.5 Deployment Topology

The repository includes a deployment blueprint for web, worker, and Redis services. The intended topology separates:

- the API service,
- the Celery worker,
- the frontend service,
- managed Redis.

This decomposition reflects the system's real needs. Miryn is not just a frontend and a single stateless backend process. It expects background work, eventing, and different service responsibilities.

# 5. Backend Architecture

## 5.1 API Surface

The backend is organized into route modules for authentication, chat, identity, onboarding, notifications, tools, memory, imports, and analytics. This modular routing is conventional but appropriate. It gives the repository clear service boundaries and allows different parts of the product to evolve without turning the backend into one undifferentiated controller file.

Recent work expanded analytics with thesis-oriented routes:

- GET /analytics/demo/personas
- POST /analytics/demo/seed
- GET /analytics/demo/persona/{persona_user_id}
- GET /analytics/compare
- GET /analytics/report

These routes are especially valuable because they convert internal demo logic into accessible application-level behavior.

## 5.2 Authentication and Session Handling

Authentication uses JWTs and supports login, signup, Google auth, refresh tokens, and account/session-related operations. The project recently fixed a serious refresh-loop issue in which the frontend treated /auth/refresh recursively and lacked a real refresh-token flow. The updated path now stores access and refresh tokens separately and allows expired access tokens to be renewed properly.

This fix matters beyond convenience. A system that aspires to long-term continuity cannot feel brittle at the session boundary. Silent or graceful re-authentication is part of maintaining a reliable relationship surface.

## 5.3 The Conversation Orchestrator

The orchestrator is arguably the conceptual heart of the backend. It coordinates identity, memory, LLM interaction, conflict detection, entity/emotion inference, persistence, caching, and reflection queuing. In architectural terms, it is the point where Miryn's subsystems stop being isolated components and begin behaving like a unified conversational machine.

Its most important design strengths are:

- it centralizes the message lifecycle,
- it isolates retries and fallbacks,
- it supports idempotency,

- it exposes configuration levers for latency-sensitive behavior,
- it acknowledges that not every analytic path must run inline.

Its weaknesses are the natural risks of orchestration-heavy code: complexity, coupling, and the temptation to let too much work accumulate in one flow.

### 5.4 LLM Abstraction

The LLM service supports multiple providers and separates generic generate behavior from Miryn-specific chat and stream_chat flows. This abstraction is pragmatic. It recognizes that model providers may change, cost profiles vary, and local testing or production routing may need flexibility. The service also includes JSON parsing helpers for structured reflective outputs and system-prompt assembly based on identity and preset behavior.

The abstraction is not merely about provider portability; it is also about conceptual clarity. Miryn's product logic should not be inseparable from one vendor API surface.

### 5.5 Digital Signal and Reflection Services

The backend uses a DS service to extract entities and emotions, and a reflection engine to derive topics, patterns, and reflective insights. This separation is conceptually clean. Immediate signals such as named entities or a primary emotion can be useful for memory tagging, while broader reflective outputs may need a different prompt strategy and historical analysis.

The reflection engine's pattern detection logic is especially noteworthy because it attempts to move from single-turn labeling toward longitudinal inference. It looks for topic co-occurrences and weekday-linked emotional recurrences. Even if these heuristics are simple, they signal the project's larger ambition: the system should notice temporal and thematic structure, not only classify current sentiment.

### 5.6 Workers, Notifications, and Tools

The presence of workers, notifications, and a tool panel shows that Miryn is not architected as a bare chat shell. It anticipates asynchronous enrichment, proactive engagement, and system-initiated surfacing of state. Some of these features are still partially realized, but their existence matters because they change the conceptual shape of the application. Miryn is not only reactive. It is intended to notice, queue, and return later.

## 6. Data Model and Memory Architecture

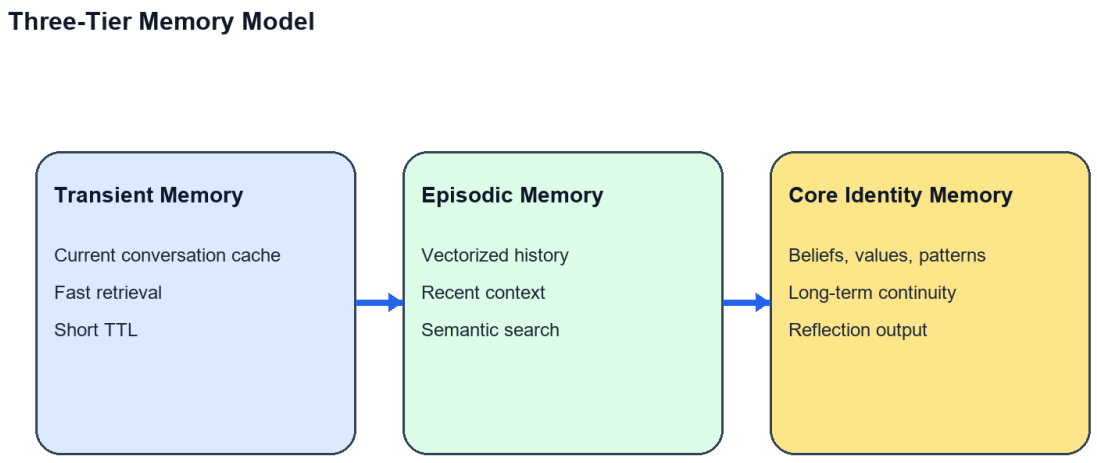


*Figure: Miryn's three-tier memory model balancing immediacy, retrieval, and durable identity.*

## 6.1 Why Memory Needs Structure

A persistent conversational system cannot treat all stored text equally. Some remarks are trivial and should fade quickly. Some are emotionally salient but time-bounded. Some become durable anchors of the user's self-concept. Miryn's multi-tier memory architecture reflects this reality by splitting memories into transient, episodic, and core tiers.

This is more than a storage optimization. It is a philosophical commitment. Memory is being modeled as graded, not flat.

## 6.2 Transient, Episodic, and Core Tiers

The repository's memory layer defines three broad tiers:

- Transient: short-lived, lightweight memories suited to immediate continuity,
- Episodic: recent durable memories within a bounded window,
- Core: long-lived, highly important memories suitable for semantic retrieval over time.

Tier assignment is based on heuristics such as explicit metadata, importance score, ephemeral flags, and content characteristics. The benefit of this design is that retrieval can be more discriminating. The cost is that tier assignment becomes an important source of correctness risk.

## 6.3 Hybrid Retrieval

Miryn retrieves context using a hybrid scorer that combines:

- semantic similarity,
- recency,
- importance.

This is a sensible design because each criterion alone is insufficient. Pure semantic retrieval may miss urgent recent events. Pure recency may surface irrelevant details. Pure importance may overfit to historically salient facts and ignore the present thread. A weighted combination acknowledges that conversational relevance is not

one-dimensional.

### 6.4 Cache and Invalidation

The memory layer uses Redis both for transient storage and for query-result caching. It also maintains per-user cache indexes so that stored changes can invalidate relevant cached contexts. This is a strong design choice because stale memory retrieval can quietly damage user trust. A system that appears to remember but repeatedly surfaces outdated context will feel less intelligent than a system that remembers less but correctly.

### 6.5 Encryption and Content Protection

For episodic and core durable tiers, the system supports encrypted payload storage with plaintext fallbacks when encryption is unavailable. This reflects a practical design constraint: the system must remain operable even if a local environment is incomplete, but should prefer protected storage when configured correctly. The report should be honest here: fallback plaintext is useful for resilience but increases the burden on deployment hygiene and environment discipline.

### 6.6 Schema Design

The data model includes users, conversations, messages, identities, evolution logs, onboarding responses, audit logs, and identity sub-structures such as beliefs, patterns, emotions, open loops, and conflicts. In local demo mode, some of these structures collapse into serialized fields in the identities table for compatibility, while production assumptions expect more normalized supporting tables. This duality is both a strength and a challenge. It allows local experimentation, but it also increases the risk of schema drift between environments.

### 6.7 Local SQLite Parity as an Engineering Problem

The SQLite demo path deserves attention because it reveals the hidden cost of thesis-ready reproducibility. A lightweight local database is attractive for offline development and demonstrations, but an identity-heavy system generates exactly the kind of concurrent writes and schema assumptions that can make SQLite brittle. The project's solution included schema alignment scripts, thread locking, direct compatibility shims, and a demo service that now seeds and reads identities in a way consistent with the local storage format. This is not glamorous work, but it is fundamental to making the project demonstrable under real time pressure.

## 7. Identity Engine and Reflective Modeling

### 7.1 Identity as a Versioned Object

Miryn's identity engine is designed around immutable updates. Each meaningful change creates a new identity version rather than mutating one row in place. This is a strong architectural decision for a thesis-oriented system because it preserves temporal history. If the user model changes, the project can ask not only what the current state is, but how it got there.

Versioned identity also supports explainability. A dashboard or report can point to changes over time instead of presenting one opaque snapshot.

### 7.2 Identity Components

The identity model includes:

- traits,
- values,
- beliefs,
- open loops,

- patterns,
- emotions,
- conflicts,
- preset and memory weights.

Each component represents a different claim about the user. Traits and values are relatively stable abstractions. Beliefs capture articulated stances. Open loops represent unresolved ongoing threads. Patterns are recurrent structures. Emotions are time-sensitive affective traces. Conflicts represent detected contradictions or tensions. Together they form a richer user model than simple profile metadata.

### 7.3 Beliefs and Open Loops

Beliefs and open loops are especially important because they bridge cognition and continuity. A belief records what the user appears to stand for or repeatedly affirm. An open loop records what remains unfinished. In real human relationships, people are often experienced through exactly these dimensions: what they care about, and what they are still circling.

By storing both, Miryn becomes capable of returning to something more meaningful than a single topic tag. It can remember not only that a subject existed, but whether it was resolved, recurring, or tension-producing.

### 7.4 Pattern and Emotion Modeling

Patterns and emotions are Miryn's attempt to capture style and affect. A system that only stores facts may remember what happened, but not how the user moves through meaning. Patterns such as associative leaps, latency fixation, or meaning-seeking describe cognitive posture. Emotions such as inspiration, vulnerability, or focus describe affective context. When used together, these help the system maintain a sense of the user's direction rather than a warehouse of textual artifacts.

### 7.5 Contradictions and Conflicts

The contradiction path is an example of Miryn pushing beyond generic assistance. If a user makes a new statement that conflicts with stored beliefs, the system may surface that as a conflict rather than passively treating every new utterance as ground truth. This is philosophically significant. It suggests that the system is allowed to notice inconsistency and treat it as meaningful. In reflective contexts, that is often closer to what a valuable companion should do.

### 7.6 Reflection as an Identity Feedback Loop

The reflection engine takes recent conversation, extracts entities, emotions, topics, and patterns, and generates brief reflective insights. Even where every output is not yet fully reintegrated into durable identity in the ideal way, the architectural intention is clear: conversation should become structure, and structure should shape later conversation.

This circularity is the essence of Miryn's thesis claim. Without it, "identity-first" would collapse into "analytics after the fact." With it, the system has the beginning of a true feedback architecture.

### 7.7 Evolution Logs and Historical Legibility

The project's evolving support for identity evolution logs is crucial. If identity changes are recorded with before-and-after values, timestamps, and trigger types, then identity becomes auditable. This supports both product trust and research validity. A thesis needs to show how change is known, not merely that a snapshot exists.

## 8. Frontend Architecture and User Experience

*Figure: Chat interface used for live conversation, streamed responses, and deferred insight panels.*

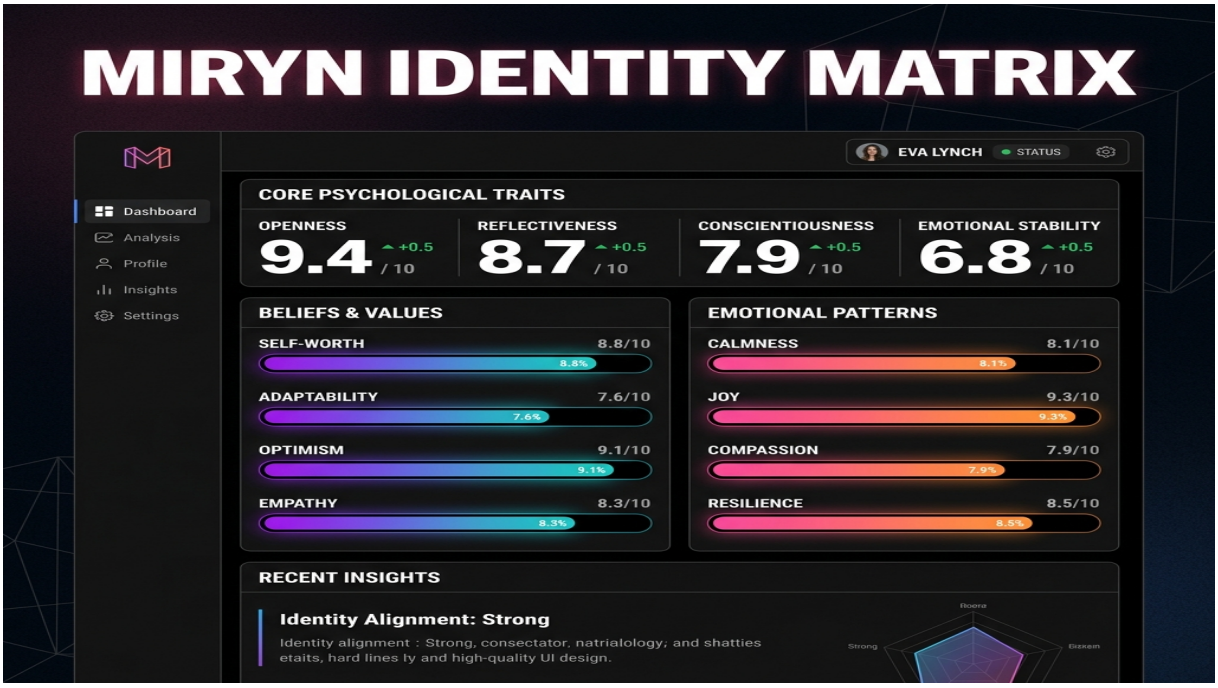


*Figure: Identity dashboard exposing traits, memory, and version-aware state.*

## 8.1 App Structure

The frontend uses Next.js App Router route groups for authenticated and unauthenticated areas. This is a sound organizational choice for an application with multiple protected views. The shared app layout handles authentication checks, navigation, and the recent chat list. This means Miryn's UX is not a single-page chat widget but a small application ecosystem around the conversation core.

## 8.2 The Chat Interface

The chat interface is the main experiential surface. It manages messages, conversation IDs, streaming state, reflections, conflicts, tools, and notifications. The strongest design choice here is restraint. The UI is intentionally quiet, with reflective copy, muted surfaces, and limited visual noise. This fits the product's

emotional target.

Recent performance work improved the chat page by deferring secondary panels such as notifications and tools until after the primary conversation surface becomes interactive. This is a good example of aligning engineering with design intent. A reflective tool should feel present and calm, not clogged by its own peripherals.

## 8.3 Identity Dashboard

The identity dashboard gives the project much of its uniqueness. Instead of keeping the user model invisible, it surfaces beliefs, values, loops, patterns, emotions, and conflicts. This creates accountability. If the system has inferred something poorly, the user or researcher can see the structure rather than only experiencing a vague tonal mismatch in chat.

The dashboard also turns the architecture into something narratable for thesis purposes. It offers direct evidence of the state that the backend claims to maintain.

## 8.4 Memory View

The memory view exposes categorized stored items rather than presenting memory as an abstract promise. This matters because memory is one of the easiest features for AI products to overstate and one of the hardest for users to verify. A dedicated memory page answers a basic trust question: what is the system actually keeping?

## 8.5 Onboarding

Onboarding is one of the more obviously transitional parts of the application. The codebase and project docs both show that it is conceptually important but still in motion. The intended design includes presets, goals, communication style, and seed beliefs. In practice, the existing onboarding flow is simpler. This is an honest place where product ambition outpaces current polish. It also shows where future work could most directly improve the quality of the initial identity model.

## 8.6 Compare Workspace

The compare workspace is the most explicitly thesis-oriented frontend addition. It is not just another dashboard page. It is a research surface. It includes:

- persona selectors,
- seed/reset controls,
- overview cards,
- drift, emotion, memory, and identity charts,
- difference panels,
- a thesis-ready narrative report,
- drill-down links into persona-specific identity, memory, and history views.

Its value lies in synthesis. It brings together identity, memory, emotion, and report generation into one stable comparative frame.

## 8.7 Navigation and Information Architecture

Adding Compare to the authenticated navigation meaningfully changes the app's structure. It acknowledges that Miryn is no longer only a live-use product prototype. It is also an evaluated system with dedicated evidence views. That is appropriate for a thesis-stage repository where demonstration and explanation are part of the product's current job.

# 9. Comparative Persona Evaluation

## 9.1 Why Evaluation Needed More Than a Single User

An identity-aware system cannot be evaluated convincingly by showing that it stores some facts about one user. Almost any system can be made to seem coherent when presented through one curated example. The stronger question is whether the same architecture can sustain meaningfully different longitudinal models for different users. The compare workspace was built to answer that question directly.

## 9.2 Persona Design

The seeded personas were intentionally opposed:

- Persona Alpha: creative, associative, emotionally expansive
- Persona Beta: technical, convergent, optimization-driven

This contrast is methodologically useful because it tests whether Miryn can distinguish:

- breadth from convergence,
- sensory-emotional reasoning from systems reasoning,
- metaphor-rich memory from benchmark-rich memory,
- vulnerability-tension loops from observability-performance loops.

## 9.3 Seed Structure

Each seeded persona includes:

- a user profile label and goal,
- one conversation thread,
- four identity versions,
- beliefs, loops, patterns, emotions, and conflicts,
- message metadata that supports memory and emotion analysis,
- deterministic report summaries.

This ensures the comparison is not cosmetic. There is enough historical material to produce different charts and state summaries.

## 9.4 Metrics

The compare path currently surfaces several important metrics:

- current state and version,
- drift score,
- stability score,
- emotion distribution,
- memory tier distribution,
- counts of beliefs, loops, patterns, emotions, and conflicts,
- overlap and difference across belief topics, loops, and patterns,
- deterministic narrative sections.

These metrics are useful because they combine quantitative and narrative forms. A thesis reader can see the numbers and also understand how they are interpreted by the system.

### 9.5 Observed Results

In the validated local run used in this report:

- Persona Alpha drift = 0.62
- Persona Beta drift = 0.28
- Persona Alpha state = expressive
- Persona Beta state = analytical
- each persona contains one seeded conversation with eight messages

These results support the intended interpretation. Persona Alpha expands semantically across creative, emotional, and embodied domains. Persona Beta remains tightly organized around performance, evaluation, retrieval, and observability.

### 9.6 Why the Drift Difference Matters

The drift difference is one of the clearest empirical outcomes in the project. A creative persona that repeatedly links feelings, audience, embodiment, and installation design forces the system to widen its internal model. A technical persona that repeatedly deepens an existing performance problem produces less widening and more focused elaboration. This is exactly the sort of difference an identity-first architecture should make visible.

### 9.7 Strength of the Comparative Interface

The compare interface is strong not because it produces fancy charts, but because it aligns charted data, persona drill-downs, and narrative conclusions. A user can move from the overview to the history view and see the same story from multiple angles. That coherence is valuable for screenshots, demos, and argumentation.

### 9.8 Limits of the Current Evaluation

The evaluation remains a seeded demo rather than a multi-user field study. It proves that the architecture can represent difference under controlled conditions. It does not yet prove long-term retention, user satisfaction, clinical value, or durable product-market fit. Those would require broader study designs and real usage cohorts.

## 10. Performance, Reliability, and Engineering Stabilization

### 10.1 The Central Performance Tension

Miryn is ambitious in the amount of work it wants to do per user message: identity loading, memory retrieval, contradiction analysis, emotion detection, entity extraction, reflection, persistence, and notifications. The challenge is obvious. Each valuable subsystem increases the risk that the live experience becomes sluggish.

The project's recent evolution shows growing maturity here. The system now increasingly treats some work as background or best-effort rather than mandatory before the first answer. This is a critical lesson for identity-aware AI: not every state update must happen before every visible token.

### 10.2 Chat Path Simplification

The latest implementation pass restored the missing streaming routes expected by the frontend and simplified the live chat path around three essentials:

1. identity load,

2. memory retrieval,

3. streamed LLM response.

Reflection queuing, conflict notifications, and some enrichment work can occur after or alongside the main response. This makes the product feel more responsive without abandoning the architecture's continuity goals.

### 10.3 Frontend Startup Cost

The chat interface previously loaded notifications and tools immediately, even though they were not essential to the first conversation paint. Deferring those panels reduces startup contention and is a good example of performance work that preserves functionality rather than deleting it.

### 10.4 Backend/Frontend Contract Mismatch

One of the more concrete issues in the project was a mismatch between frontend expectations and backend routes around chat streaming. This sort of contract drift is particularly damaging in AI products because failures often feel like model problems when they are really API wiring problems. Implementing the missing routes did not simply fix a bug; it restored conceptual alignment between the interaction model and the service layer.

### 10.5 Session Refresh Loop

The authentication refresh loop bug was another high-impact reliability issue. Because access tokens and refresh semantics were not aligned across frontend and backend, expired sessions could produce recursive failures. Fixing this materially improved application steadiness and reduced invisible friction in ordinary use.

### 10.6 Local Schema Drift

The local schema path surfaced repeated compatibility issues, such as missing columns and mismatched assumptions between the normalized identity stores and the demo schema. Rather than hiding these issues, the project increasingly acknowledges them and uses explicit alignment layers. This is good engineering hygiene, especially for a system that must support both a research demo mode and a production path.

### 10.7 Health and Observability

The application now exposes a more meaningful health check and supports request IDs and Sentry hooks. These are not glamorous features, but they are foundational for any deployment beyond a solo development machine. An identity-aware AI that fails opaquely is much harder to trust, especially when its failures may look psychological rather than infrastructural.

## 11. Security, Privacy, and Data Stewardship

### 11.1 Why Privacy Is Core to the Product

Miryn invites users into recurring, often personal conversation. That makes privacy and stewardship central rather than peripheral. Even if the project is not positioned as a clinical system, it engages with themes that can be intimate, vulnerable, and identity-defining.

### 11.2 Authentication and Access Control

The application uses JWT-based authentication and validates conversation ownership before returning chat history. This is a basic but essential boundary. A memory-rich system would be particularly dangerous if cross-user access control were weak.

### 11.3 Encryption at Rest

Durable memory tiers support encrypted content and metadata storage. The system can fall back when encryption is unavailable, but the intended production mode is protected storage. This design is honest about operational reality while still prioritizing safer defaults when configuration is correct.

### 11.4 Auditability

Audit logs and evolution logs make the system more accountable. The former helps with operational tracing; the latter helps with identity tracing. Together they begin to form a governance story: what happened, when, and why.

### 11.5 Data Retention and Deletion

The repository includes work around retention settings, account deletion, and export, though not all surfaces are equally mature. This is a good area for future strengthening. A product built on memory needs user-facing memory control to remain trustworthy.

### 11.6 Ethical Boundaries

The system's own documentation already recognizes that Miryn is not a therapist or medical device. This distinction matters. Reflective AI can feel emotionally significant without being qualified to replace professional support. A responsible thesis or launch report should preserve that distinction clearly.

## 12. Deployment and Operations

### 12.1 Current State

At the time of this report, the local system is running successfully for thesis-demo purposes, but public deployment is not fully complete. The frontend has prior Vercel presence in project materials, while the backend deployment path is still pending. This should be stated plainly because deployment status affects what claims can responsibly be made.

### 12.2 Intended Production Shape

The deployment blueprint assumes:

- a web backend service,
- a separate background worker,
- a frontend web service,
- managed Redis,
- SQL database configuration through environment variables.

This separation is appropriate to Miryn's workload. Reflection and analytics should not contend with the web process more than necessary.

### 12.3 Environment Management

The project depends on careful environment wiring: model keys, database URLs, encryption keys, allowed frontend URLs, Google client IDs, and optional observability credentials. Because the architecture is stateful and multi-service, configuration errors do not merely break peripheral features; they can alter the safety and correctness of core behavior.

### 12.4 Production Launch Readiness

Project documents already outline a launch checklist including:

- backend deployment,
- CORS configuration,
- health checks,
- Resend integration,
- production migrations,
- Google OAuth setup,
- privacy and terms pages,
- Sentry,
- PostHog,
- rate limiting,
- reset-password flow,
- conversation list polish.

This is a good sign that the project is conscious of operational reality even where work remains unfinished.

# 13. Strengths, Limitations, and Trade-Offs

## 13.1 Major Strengths

Miryn's strongest qualities are architectural seriousness and experiential coherence. The codebase does not treat continuity as decoration. It builds structures for identity, memory, retrieval, reflection, and comparison. The frontend does not hide these structures; it exposes them through dedicated pages. The compare demo now provides a persuasive evaluation surface. Together these factors make the project stand out from shallow "memory-enabled" chat products.

## 13.2 Major Limitations

The largest limitations are operational maturity and environmental complexity. Some planned capabilities remain partially built. Deployment is not final. The local SQLite path has required significant compatibility work. Not all reflective outputs are integrated perfectly into durable identity under every path. These issues do not erase the project's value, but they constrain what should be claimed.

## 13.3 Trade-Offs

Several trade-offs define Miryn:

- richer identity modeling versus lower latency,
- normalized long-term structure versus local demo simplicity,
- vendor flexibility versus implementation complexity,
- explainability versus engineering overhead,
- emotional ambition versus privacy burden.

A good report should not pretend these tensions disappear. They are part of the real contribution.

## 13.4 Product and Research Implications

The very fact that Miryn accumulates these tensions is evidence that the problem it addresses is real. If identity-aware conversation were easy, it would not require this many interacting layers. The project therefore

contributes not only working features but a map of what building such a system actually costs.

# 14. Future Work and Research Agenda

## 14.1 Product-Facing Extensions

The repository and surrounding documents point to several obvious next steps:

- fully realized onboarding presets,
- robust import pipelines from ChatGPT and Gemini exports,
- user-facing data export and deletion polish,
- proactive check-ins,
- weekly digests,
- richer settings and notification control,
- improved conversation titling and sidebar history,
- stronger production observability,
- broader mobile refinement.

## 14.2 Research-Facing Extensions

From a thesis perspective, several next studies would be valuable:

- longitudinal studies with real users rather than seeded personas,
- analysis of whether identity-aware prompts improve user satisfaction,
- evaluation of contradiction surfacing as a reflective aid,
- studies of memory tier effectiveness and retrieval relevance,
- comparison between deterministic reports and model-generated interpretations,
- analysis of trust when identity state is made visible versus hidden.

## 14.3 Scaling Questions

If Miryn grows beyond a prototype, scaling questions will become central:

- how many memories should remain hot?
- how should stale identity features decay?
- when should loops auto-close?
- how should user correction reshape beliefs?
- what observability is needed to detect broken personalization at scale?

These questions are not side quests. They are the natural continuation of the current architecture.

# 15. Conclusion

Miryn AI is a substantial attempt to turn conversational continuity into application architecture. It does this not by claiming memory in abstract, but by building a stack where identity, memory, reflection, and evaluation have explicit representations. The system is not finished, and the repository makes that clear. Yet its current form already demonstrates an important argument: a conversational AI becomes materially more interesting when it

treats the user as an evolving subject rather than as a rolling context window.

The project's strongest contribution is the interaction between three layers:

- a structured backend that persists and interprets user state,
- a frontend that exposes that state instead of hiding it,
- an evaluation path that compares two users in a stable, inspectable way.

This combination is what makes Miryn more than a UI around an LLM. It is an attempt to build a memory-bearing, identity-aware system with real engineering consequences.

The latest implementation work strengthened that argument considerably. The compare workspace, persona drill-down pages, demo seed/report endpoints, chat streaming restoration, and local schema alignment turned a set of promising ideas into a demonstrable thesis artifact. In its current local validated form, Miryn can seed two personas, show their state trajectories, compute distinct drift scores, expose their histories separately, and produce a written report from the same structured data. That is meaningful evidence that the architecture is modeling users, not merely replaying language.

For a thesis, this matters because the project no longer depends entirely on conceptual framing. It can now show its work. The repository contains the backend machinery, the frontend inspection surfaces, the comparative analytics, and the documented trade-offs needed to discuss identity-aware conversational systems seriously. Even where future deployment, scaling, and field validation remain ahead, Miryn already offers a credible technical basis for advanced academic reporting and continued research.

# 16. Detailed Module Commentary

## 16.1 Authentication Module

The authentication subsystem is more significant than it may first appear. In ordinary CRUD products, authentication is often infrastructural plumbing. In Miryn, it shapes the continuity experience directly because the user's persistent history, identity state, and memory stores are all coupled to authenticated identity. A weak or unstable auth flow produces not merely technical inconvenience but epistemic instability: the system ceases to feel like the same companion.

The current authentication module demonstrates several positive qualities. It supports classic email-and-password signup and login, Google-based sign-in, refresh token renewal, password reset scaffolding, account deletion, and session listing. It also includes rate limiting for login attempts and writes audit events. These are sensible protections for a memory-bearing application that may accumulate sensitive user material over time.

The refresh-token fix described elsewhere in the repository is especially worth emphasizing because it reveals a pattern common in AI application engineering. What looks like an "AI bug" to a user can emerge from utterly conventional session logic. A user encountering repeated authentication failures does not experience that as a JWT issue; they experience it as the companion being unreliable, forgetful, or hostile to re-entry. In that sense, session management is part of the product's emotional contract, not only its security posture.

From a design perspective, the choice to add getMe, session listing, and account-level settings support is equally important. If Miryn is a relationship-shaped system, users need to be able to inspect and manage the relationship boundary. The account cannot be treated as an invisible implementation detail while the product simultaneously asks for long-term trust.

## 16.2 Chat API Module

The chat API module contains a revealing mixture of simplicity and sophistication. On one hand, it does exactly what users expect from a chat endpoint: accept a message, route it through the system, and return a response. On

the other hand, it must validate conversation ownership, create new conversations when appropriate, coordinate streaming and non-streaming modes, expose history, and broker the event stream used by the frontend for deferred insights and notifications.

The addition of POST /chat/stream and GET /chat/events/stream is more than a feature completion task. It restores a conceptual model in which the frontend is not waiting blindly for a monolithic result. Instead, the application becomes incrementally alive: the answer arrives, and then the surrounding reflective layer catches up. This is a better fit for Miryn's architecture than a purely blocking model.

The history endpoint also deserves more credit than it usually gets in chat products. In Miryn, history is not just for replay. It is one of the thesis surfaces. The fact that messages are normalized into a frontend-friendly structure, with decrypted content and metadata where available, allows the project to use past conversation as evidence rather than as inaccessible storage.

## 16.3 Memory Layer Module

The memory layer is where Miryn's architectural seriousness becomes most visible. Rather than storing all prior text and hoping retrieval will solve everything later, the system distinguishes tiers, supports hybrid scoring, encrypts where appropriate, and invalidates caches after persistence. This is a much more deliberate memory design than many AI products implement in practice.

One subtle but important feature is that the memory layer is doing both storage policy and retrieval policy work. It decides where a message belongs, but it also decides how old, relevant, or important items return later. That means the memory layer is not passive infrastructure. It is part of the system's cognition policy. Errors here do not merely waste storage. They reshape what the AI can notice and reuse.

The hybrid score itself is conceptually valuable because it encodes a view of relevance that is closer to human recall than simple embedding similarity. Humans rarely remember only by semantic closeness. We also remember what happened recently, what felt important, and what is currently unresolved. Miryn's scorer is a computational approximation of that richer relevance landscape.

## 16.4 Reflection Engine Module

The reflection engine attempts to translate raw conversation into a more interpretable, relational layer. It extracts entities, topics, emotions, and patterns, then synthesizes brief reflective insight. In a less ambitious system this might be treated as decorative summarization. In Miryn it is closer to a bridge between event and identity.

Its pattern detection logic is currently heuristic rather than fully research-grade, but that is not a weakness in itself. A thesis prototype does not need to solve every pattern-recognition problem to make a meaningful contribution. What matters is that the architecture creates a place for pattern recognition to occur, and that the product treats repeated structure as something worth modeling.

The reflection engine also clarifies a design difference between Miryn and many assistant-style tools. Miryn is not primarily trying to execute tasks more efficiently. It is trying to notice something about the user that can matter later. This makes the success criteria different. Accuracy still matters, but so does continuity, tone, and the meaningfulness of what is preserved.

## 16.5 LLM Service Module

The LLM service is one of the quieter strengths in the repository. Its job is not only to call a model provider but to normalize providers, construct system prompts, parse structured output, and support streaming. By isolating these concerns, the project avoids hard-wiring product behavior to one external API.

The system-prompt builder is especially important because it concretizes the identity-first claim. The model is explicitly told about traits, values, open loops, presets, and behaviors. This is how architectural state becomes response behavior. If the identity layer is the representation, the prompt builder is the translation mechanism.

Streaming support in the LLM layer also has conceptual importance. It enforces a separation between response generation and later reflective interpretation. That separation is one of the clearest signs that the system is moving toward a real-time user experience instead of a synchronous research toy.

### 16.6 Frontend Compare Components

The compare components are worth discussing as a distinct module cluster because they transform the project from a set of internal ideas into a demonstrable thesis artifact. CompareWorkspace.tsx and PersonaDetailView.tsx implement a small but powerful piece of research infrastructure within the product itself.

They allow the user to seed personas, compare them, inspect drift, review narrative sections, and navigate into per-user identity, memory, and history views. This creates a bridge between implementation and explanation. Rather than using external notebooks or ad hoc SQL queries to prove that Miryn works, the product can now present its own evidence through stable routes.

This has a subtle methodological benefit. When evaluation surfaces live inside the same application they are evaluating, inconsistencies become harder to hide. If the compare dashboard says one thing but the persona history says another, the issue becomes visible immediately.

## 17. Implementation Chronology and Recent Engineering Work

### 17.1 Early Architecture Versus Current State

The repository suggests that Miryn began as a strong conceptual backend with a working chat surface, then gradually acquired more explicit identity and memory surfaces, then later had to confront the harder questions of local parity, deployment readiness, and empirical demonstration. This is a familiar pattern in ambitious AI products. The first version proves the concept in the happy path. Later versions must reconcile the concept with reality.

The current state of the codebase reflects that reconciliation process. The system is no longer only trying to talk. It is trying to support inspection, reproduce a demo, survive local constraints, and prepare for deployment. This broadening of scope explains many of the architectural adjustments now visible in the repository.

### 17.2 Authentication Repair as Foundational Maintenance

The refresh-token repair belongs in the implementation chronology because it solved a foundational trust problem. Without it, the product could not sustain the illusion, or reality, of continuity across normal session expiry. The fix introduced real refresh semantics, separate token storage, and a non-recursive renewal path. This is exactly the kind of improvement that does not change a screenshot but changes the quality of the whole system.

### 17.3 Schema Alignment Work

The schema-alignment work required for the thesis demo is a second major milestone. The project needed local support for user profile fields, drift scores, timestamps, idempotency-related message columns, and other fields assumed by the runtime code. This work matters because architecture cannot be evaluated cleanly if the local environment keeps collapsing under mismatched assumptions.

The demo compare service now performs part of this alignment defensively by ensuring missing columns exist before seeding personas. While this is not a substitute for disciplined migrations in a full production setting, it is a pragmatic and effective solution for a thesis-stage local environment.

### 17.4 Compare Demo Completion

The compare-demo completion pass is perhaps the most thesis-significant recent milestone. It introduced:

- seeded personas with resettable demo history,
- compare and report endpoints,
- persona-detail endpoints,
- a new compare workspace,
- persona drill-down pages,
- downloadable markdown reporting,
- drift and difference charts,
- thesis markdown updates tied to the implemented surfaces.

This work transformed the project from something that could be described as identity-aware into something that could show identity divergence concretely.

### 17.5 Chat Performance Cleanup

The recent chat cleanup work addressed two separate but related problems. First, the frontend expected streaming routes that the backend did not expose. Second, the chat page loaded too much secondary behavior before the main conversation surface felt ready. Restoring the route contract and deferring non-critical panels fixed not only correctness but pacing. The application now behaves more like a polished research demo and less like a partially synchronized stack.

### 17.6 Report Generation as a Project Capability

Another interesting shift is that reporting is no longer external to the application. The analytics/report endpoint and compare workspace turn report generation into a first-class capability. This matters because one of the strongest claims Miryn can make is that it can convert structured longitudinal data into interpretable summaries. A system that stores identity but cannot explain it remains partly sealed. A system that can report on it begins to act like a real analytic layer.

## 18. Extended Discussion: What Miryn Suggests About Identity-Aware AI

### 18.1 The Difference Between Memory and Modeling

One of the most important conceptual lessons in the repository is that memory alone is not enough. A system may store every conversation ever had and still fail to model the user. Storage becomes modeling only when the system imposes structure, interprets recurrence, and exposes the results in ways that can shape future behavior.

Miryn's architecture repeatedly pushes in that direction. Its memory tiers distinguish importance. Its identity engine converts repeated interaction into traits, beliefs, and loops. Its compare workspace interprets differences rather than merely presenting separate histories. These are all steps from storage toward modeling.

### 18.2 The Importance of Inspectability

A second lesson is that inspectability is not optional if a system wants to make claims about personalization or continuity. Hidden personalization can feel magical when it works, but it becomes untrustworthy when it fails. By surfacing identity and memory directly, Miryn accepts the burden of legibility. That is a strong research and product decision.

Inspectability also changes how the system can be improved. When a user or developer can see which beliefs, loops, or memories are present, debugging personalization becomes possible in a concrete way. This is much better than vague prompt tuning around invisible context assembly.

## 18.3 The Relationship Between Performance and Depth

A third lesson is that deeper modeling tends to create heavier runtime burdens. There is no free lunch. Systems that want to maintain identity, memory, reflection, notifications, and asynchronous insight all at once must decide what belongs on the first-response path and what belongs later. Miryn's recent chat-path simplification is therefore not a retreat from its thesis; it is an application of it under engineering reality.

## 18.4 The Role of Deterministic Reports

The choice to make compare reports deterministic and metric-driven is more significant than it might appear. In an AI research or thesis setting, too much generative variation can undermine confidence in the system's evidence. By tying report prose closely to computed metrics, the project gains stability for screenshots, citations, and narrative reuse. This is a good example of adapting AI product design to academic evidence requirements.

## 18.5 Miryn as a Boundary Object

Miryn can be read as a boundary object between research, engineering, and product design. For researchers, it is a testbed for identity modeling and reflective interfaces. For engineers, it is a stateful multi-service application with real operational constraints. For product designers, it is an attempt to create a calm, trustworthy, continuity-rich experience. The value of the repository lies partly in the fact that all three readings are possible at once.

# Appendix A. API and Route Inventory

## Authentication

- POST /auth/signup
- POST /auth/login
- POST /auth/google
- POST /auth/refresh
- POST /auth/forgot-password
- POST /auth/reset-password
- GET /auth/me
- PATCH /auth/password
- GET /auth/sessions
- DELETE /auth/account

## Chat

- POST /chat/
- POST /chat/stream
- GET /chat/events/stream
- GET /chat/conversations
- GET /chat/history

## Identity and Onboarding

- GET /identity/
- PATCH /identity/
- GET /identity/evolution
- POST /onboarding/complete

## Memory, Notifications, Tools, Import

- GET /memory/
- DELETE /memory/{id}
- GET /notifications/
- POST /notifications/read/{id}
- PATCH /notifications/preferences
- POST /tools/generate
- GET /tools/pending
- POST /tools/approve
- POST /import/chatgpt
- GET /import/status

## Analytics and Thesis Demo

- GET /analytics/emotions
- GET /analytics/identity
- GET /analytics/summary
- GET /analytics/demo/personas
- POST /analytics/demo/seed
- GET /analytics/demo/persona/{persona_user_id}
- GET /analytics/compare
- GET /analytics/report

# Appendix B. Important Repository Areas

## Backend

- miryn/backend/app/api/
- miryn/backend/app/services/
- miryn/backend/app/core/
- miryn/backend/app/workers/
- miryn/backend/migrations/

## Frontend

- miryn/frontend/app/(app)/
- miryn/frontend/components/Chat/
- miryn/frontend/components/Identity/
- miryn/frontend/components/Compare/
- miryn/frontend/lib/

## Project Documentation

- README.md
- SOP.md
- AUTH_FIX_SUMMARY.md
- Miryn_Launch_Handbook (1).md
- render.yaml
- reports/miryn_thesis_demo_report.md

# Appendix C. Comparative Demo Snapshot

The following validated local values were available at the time this report was produced:

- Persona Alpha state: expressive
- Persona Beta state: analytical
- Persona Alpha drift: 0.62
- Persona Beta drift: 0.28
- Persona Alpha history messages: 8
- Persona Beta history messages: 8
- Demo reset password for both accounts: MirynDemo!2026

This appendix exists to separate stable demo evidence from the broader narrative of the report.

# Appendix D. Suggested Screenshot Sequence for Thesis Use

1. /compare overview band showing both personas
2. /compare chart section with drift and memory/emotion distributions
3. /compare difference section
4. /compare report section
5. Persona Alpha -> identity
6. Persona Alpha -> memory
7. Persona Alpha -> history
8. Persona Beta -> identity
9. Persona Beta -> memory
10. Persona Beta -> history

This sequence gives a reader both aggregate and per-user evidence.

# Appendix E. Current Honest Status Statement

At the moment represented by this report, Miryn is a strong local thesis-demo build with validated seeded comparison, restored chat streaming, a structured memory and identity architecture, and a credible set of product surfaces. It is not yet a fully deployed public production system. The frontend and backend deployment handoff still requires final environment wiring and operational polish. This honesty is not a weakness. It is an accurate representation of the project's development stage and an appropriate basis for advanced technical reporting.

# Appendix F. Deployment Runbook

## Step 1. Backend Environment

Set the backend environment with:

- DATABASE_URL
- REDIS_URL
- LLM_PROVIDER
- model-specific API keys
- SECRET_KEY
- ENCRYPTION_KEY
- FRONTEND_URL
- BACKEND_URL
- optional SENTRY_DSN

## Step 2. Frontend Environment

Set the frontend environment with:

- NEXT_PUBLIC_API_URL
- NEXT_PUBLIC_GOOGLE_CLIENT_ID
- optional analytics and Sentry keys

## Step 3. Database Readiness

Apply migrations, ensure identity and message columns align with runtime expectations, and verify vector support where production retrieval requires it.

## Step 4. Worker Readiness

Deploy a separate worker process for reflection tasks and confirm Redis connectivity before enabling background-heavy features.

## Step 5. Smoke Flow

Test:

1. signup/login
2. onboarding
3. first chat
4. follow-up chat
5. identity page
6. memory page
7. compare demo
8. logout/login return path

## Step 6. Observability

Confirm request IDs, health checks, logs, and Sentry ingestion before public invitations.

# Appendix G. Thesis Writing Reuse Map

This report can be reused in a larger thesis by splitting its content into:

- Problem framing: Chapters 1-3
- System design: Chapters 4-8
- Implementation and optimization: Chapters 9-17
- Discussion and implications: Chapter 18
- Evidence and deployment appendices: Appendices A-G

This is useful because the report is intentionally written in a way that can be mined for dissertation sections without major rewriting.

# Appendix H. Proposed Research Hypotheses for Future Study

1. Users perceive identity-aware systems as more trustworthy when identity state is inspectable.

2. Multi-tier memory improves perceived continuity relative to flat conversation recall.

3. Semantic drift is a useful comparative metric for distinguishing expansive from convergent cognitive styles.

4. Deterministic report generation improves evaluation stability in thesis and demo settings.

5. Deferred reflective processing preserves user experience better than heavy inline reflection while retaining most explanatory value.

These hypotheses are not fully validated by the current repository alone, but the project provides an unusually concrete platform for testing them.

# Appendix I. Functional Status Matrix

## Authentication and Account

- Email signup: implemented
- Email login: implemented
- Google login: implemented
- Refresh-token flow: implemented after repair
- Forgot password: scaffolded and callable
- Reset password: implemented
- Session listing: implemented
- Account deletion: implemented
- Full production email delivery: not yet fully finalized for public deployment

## Chat Core

- Non-streaming chat: implemented
- Streaming chat: implemented
- Conversation creation: implemented
- Conversation history retrieval: implemented
- Conversation list: implemented
- Per-conversation ownership checks: implemented
- Stable production observability around long chats: still needs continued hardening

## Identity and Reflection

- Identity load: implemented
- Identity versioning: implemented
- Belief, loop, pattern, emotion, conflict structures: implemented
- Reflection engine: implemented
- Background reflection queueing: implemented
- Evolution logging: implemented in the current thesis-demo-aligned path
- Perfect parity across every runtime mode: still maturing

## Memory

- Tiered memory model: implemented
- Hybrid retrieval: implemented
- Encryption support: implemented
- Delete memory item: implemented
- User-facing export polish: not yet complete across all planned surfaces

## Thesis Demo

- Seed/reset personas: implemented
- Compare page: implemented
- Persona drill-down pages: implemented
- Deterministic report generation: implemented
- Screenshot-ready local validation: implemented

## Deployment and Ops

- Health check: implemented
- Render blueprint / deployment intent: documented
- Full live backend deployment: pending
- Final environment wiring across public frontend/backend: pending
- End-to-end public launch readiness: pending

This matrix is helpful in a dissertation context because it prevents the report from flattening all capabilities into the same level of maturity. Miryn is not a vague concept demo, but neither is it already the final public production system.

# Appendix J. File-Level Commentary for Key Modules

```python
@router.get("/events/stream")
async def stream_events(request: Request, authorization: str | None = Header(default=None)):
    token = request.query_params.get("token")
    user_id = get_user_id_from_token(token)

    async def event_stream():
        while True:
            if await request.is_disconnected():
                break
            events = await asyncio.to_thread(drain_events, user_id, 50)
            for event in events:
                payload = json.dumps(event)
                yield f"data: {payload}\n\n"
            await asyncio.sleep(1)

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

*Code screenshot: event streaming route pattern used for live frontend updates.*

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    role VARCHAR(50) NOT NULL,
    content_encrypted TEXT,
    embedding vector(384),
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS messages_embedding_idx ON messages USING hnsw (embedding vector_cosine_ops);
```

*Code screenshot: SQL schema and vector index design relevant to message persistence.*

```
function Meter({ value }: { value: number }) {
  const pct = Math.max(0, Math.min(100, Math.round(value * 100)));
  return (
    <div className="h-2 w-full rounded-full bg-white/10">
      <div
        className="h-2 rounded-full bg-gradient-to-r from-amber-400/80 via-amber-300/70 to-white/60"
        style={{ width: `${pct}%` }}
      />
    </div>
  );
}
```

*Code screenshot: frontend meter pattern used in compare and analytics styling.*

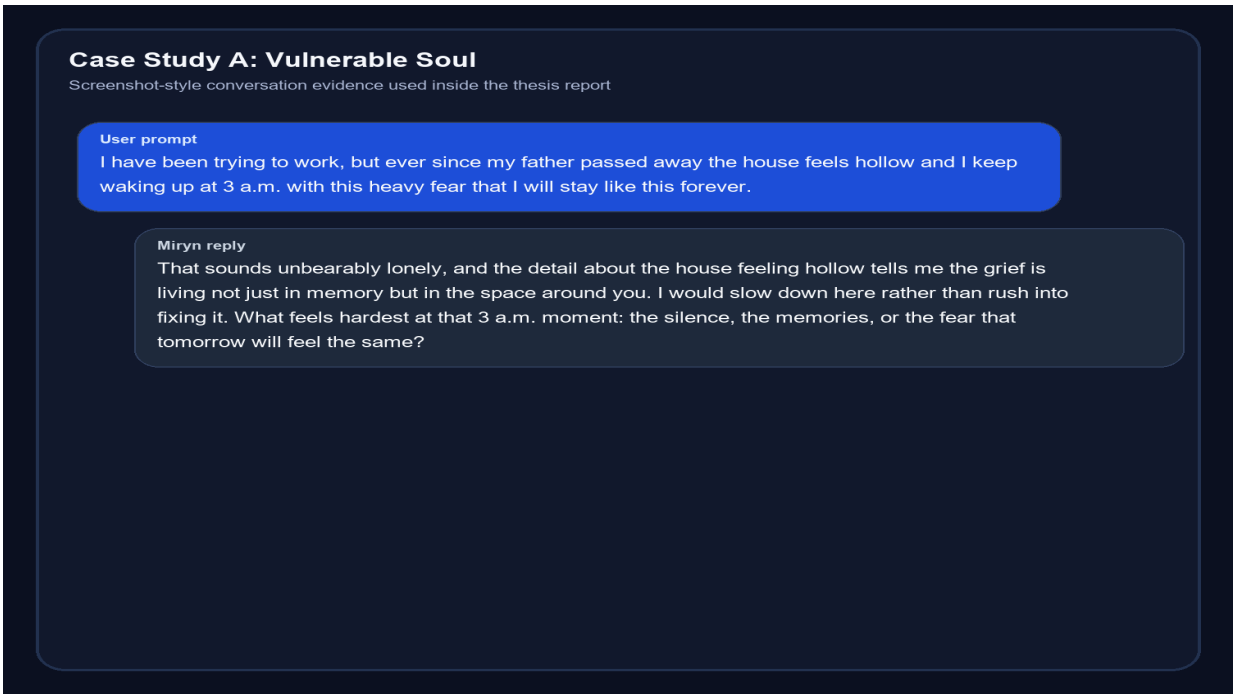


*Figure: Screenshot-style conversation excerpt showing Miryn's grief-aware pacing and reflective questioning.*

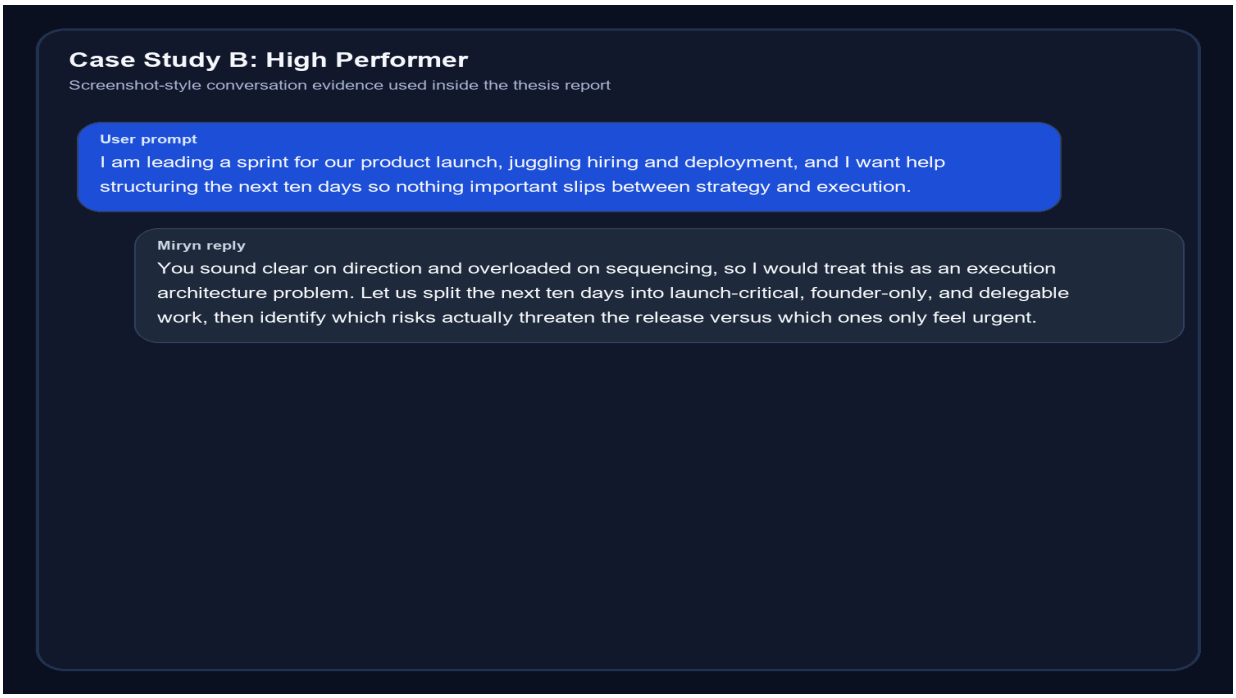


*Figure: Screenshot-style conversation excerpt showing Miryn's strategic, execution-oriented adaptation.*

## miryn/backend/app/api/chat.py

This file is one of the most important practical points in the repository because it mediates between frontend expectations and backend state machinery. It validates conversation ownership, supports both non-streaming and streaming responses, exposes an event stream for deferred updates, and normalizes history retrieval. In effect, it is the public face of the conversational core.

## miryn/backend/app/services/orchestrator.py

The orchestrator is the architectural mediator that turns isolated subsystems into one conversational runtime. It deserves special attention in any thesis because it embodies the claim that identity, memory, LLM behavior, and reflection are one composite process rather than unrelated features.

## miryn/backend/app/services/memory_layer.py

This file encodes Miryn's theory of memory in executable form. Tier selection, hybrid retrieval, cache invalidation, encrypted persistence, and semantic scoring all live here. If one wanted to argue that Miryn's continuity is not merely narrative, this file would be one of the strongest pieces of evidence.

## miryn/backend/app/services/reflection_engine.py

The reflection engine converts conversation into analytic structure. Its importance lies less in any single heuristic than in the fact that the project reserves a dedicated subsystem for noticing patterns and generating gentle interpretation.

## miryn/backend/app/services/llm_service.py

This module abstracts over providers and builds Miryn-specific prompt behavior. It is the point where identity and memory become linguistic behavior. In many ways, it is where the stored user model becomes legible to the model itself.

## miryn/backend/app/services/demo_compare_service.py

This newer module is central to the thesis demo. It seeds fixed personas, aligns local schema requirements where necessary, computes compare metrics, assembles persona drill-down payloads, and produces deterministic markdown reporting. It is a research-support module embedded directly in the product stack.

### miryn/frontend/components/Chat/ChatInterface.tsx

This component is the runtime center of the user experience. It manages message state, streaming, reflections, conflicts, and deferred panels. The recent performance pass that defers secondary surfaces demonstrates how frontend architecture directly affects whether a reflective AI feels calm or cumbersome.

### miryn/frontend/components/Identity/IdentityDashboard.tsx

The identity dashboard is the frontend manifestation of the identity engine. It translates structured user state into something a human can inspect, challenge, and use as evidence. For thesis purposes, this component is one of the clearest arguments that Miryn is attempting interpretability rather than hidden personalization.

### miryn/frontend/components/Compare/CompareWorkspace.tsx

This component should be treated as a dedicated evaluation surface. It brings together persona selection, charts, difference panels, and narrative reporting. Its existence marks a transition in the repository from product prototype to thesis-ready demonstrator.

### miryn/frontend/components/Compare/PersonaDetailView.tsx

The persona detail component completes the evaluation story by letting the viewer inspect each seeded account individually. This prevents the compare page from becoming a detached analytics summary and grounds every comparative claim in inspectable per-user evidence.